

```
MatDoub SVD::nullspace(Doub thresh = -1.){
```

Give an orthonormal basis for the nullspace of **A** as the columns of a returned matrix. `thresh` as above.

```
    Int j,jj,nn=0;
    MatDoub nullsp(n,nullity(thresh));
    for (j=0;j<n;j++) {
        if (w[j] <= tsh) {
            for (jj=0;jj<n;jj++) nullsp[jj][nn] = v[jj][j];
            nn++;
        }
    }
    return nullsp;
}
```

The meaning of the optional parameter `thresh` is discussed below.

2.6.2 SVD of a Square Matrix

We return to the case of a square $N \times N$ matrix **A**. **U**, **V**, and **W** are also square matrices of the same size. Their inverses are also trivial to compute: **U** and **V** are orthogonal, so their inverses are equal to their transposes; **W** is diagonal, so its inverse is the diagonal matrix whose elements are the reciprocals of the elements w_j . From (2.6.1) it now follows immediately that the inverse of **A** is

$$\mathbf{A}^{-1} = \mathbf{V} \cdot [\text{diag}(1/w_j)] \cdot \mathbf{U}^T \quad (2.6.6)$$

The only thing that can go wrong with this construction is for one of the w_j 's to be zero, or (numerically) for it to be so small that its value is dominated by roundoff error and therefore unknowable. If more than one of the w_j 's has this problem, then the matrix is even more singular. So, first of all, SVD gives you a clear diagnosis of the situation.

Formally, the *condition number* of a matrix is defined as the ratio of the largest (in magnitude) of the w_j 's to the smallest of the w_j 's. A matrix is singular if its condition number is infinite, and it is *ill-conditioned* if its condition number is too large, that is, if its reciprocal approaches the machine's floating-point precision (for example, less than about 10^{-15} for values of type `double`). A function returning the condition number (or, rather, its reciprocal, to avoid overflow) is implemented in SVD.

Now let's have another look at solving the set of simultaneous linear equations (2.6.5) in the case that **A** is singular. We already saw that the set of *homogeneous* equations, where **b** = 0, is solved immediately by SVD. The solution is any linear combination of the columns returned by the `nullspace` method above.

When the vector **b** on the right-hand side is not zero, the important question is whether it lies in the range of **A** or not. If it does, then the singular set of equations *does* have a solution **x**; in fact it has more than one solution, since any vector in the nullspace (any column of **V** with a corresponding zero w_j) can be added to **x** in any linear combination.

If we want to single out one particular member of this solution set of vectors as a representative, we might want to pick the one with the smallest length $|\mathbf{x}|^2$. Here is how to find that vector using SVD: Simply *replace* $1/w_j$ by zero if $w_j = 0$. (It is not very often that one gets to set $\infty = 0$!) Then compute, working from right to

left,

$$\mathbf{x} = \mathbf{V} \cdot [\text{diag}(1/w_j)] \cdot (\mathbf{U}^T \cdot \mathbf{b}) \quad (2.6.7)$$

This will be the solution vector of smallest length; the columns of $\mathbf{V}$ that are in the nullspace complete the specification of the solution set.

Proof: Consider $|\mathbf{x} + \mathbf{x}'|$, where $\mathbf{x}'$ lies in the nullspace. Then, if $\mathbf{W}^{-1}$ denotes the modified inverse of $\mathbf{W}$ with some elements zeroed,

$$\begin{aligned} |\mathbf{x} + \mathbf{x}'| &= |\mathbf{V} \cdot \mathbf{W}^{-1} \cdot \mathbf{U}^T \cdot \mathbf{b} + \mathbf{x}'| \\ &= |\mathbf{V} \cdot (\mathbf{W}^{-1} \cdot \mathbf{U}^T \cdot \mathbf{b} + \mathbf{V}^T \cdot \mathbf{x}')| \\ &= |\mathbf{W}^{-1} \cdot \mathbf{U}^T \cdot \mathbf{b} + \mathbf{V}^T \cdot \mathbf{x}'| \end{aligned} \quad (2.6.8)$$

Here the first equality follows from (2.6.7), and the second and third from the orthonormality of $\mathbf{V}$. If you now examine the two terms that make up the sum on the right-hand side, you will see that the first one has nonzero j components only where $w_j \neq 0$, while the second one, since $\mathbf{x}'$ is in the nullspace, has nonzero j components only where $w_j = 0$. Therefore the minimum length obtains for $\mathbf{x}' = 0$, q.e.d.

If $\mathbf{b}$ is not in the range of the singular matrix $\mathbf{A}$, then the set of equations (2.6.5) has no solution. But here is some good news: If $\mathbf{b}$ is not in the range of $\mathbf{A}$, then equation (2.6.7) can still be used to construct a “solution” vector $\mathbf{x}$. This vector $\mathbf{x}$ will not exactly solve $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. But, among all possible vectors $\mathbf{x}$, it will do the closest possible job in the least-squares sense. In other words, (2.6.7) finds

$$\mathbf{x} \quad \text{which minimizes} \quad r \equiv |\mathbf{A} \cdot \mathbf{x} - \mathbf{b}| \quad (2.6.9)$$

The number r is called the *residual* of the solution.

The proof is similar to (2.6.8): Suppose we modify $\mathbf{x}$ by adding some arbitrary $\mathbf{x}'$. Then $\mathbf{A} \cdot \mathbf{x} - \mathbf{b}$ is modified by adding some $\mathbf{b}' \equiv \mathbf{A} \cdot \mathbf{x}'$. Obviously $\mathbf{b}'$ is in the range of $\mathbf{A}$. We then have

$$\begin{aligned} |\mathbf{A} \cdot \mathbf{x} - \mathbf{b} + \mathbf{b}'| &= |(\mathbf{U} \cdot \mathbf{W} \cdot \mathbf{V}^T) \cdot (\mathbf{V} \cdot \mathbf{W}^{-1} \cdot \mathbf{U}^T \cdot \mathbf{b}) - \mathbf{b} + \mathbf{b}'| \\ &= |(\mathbf{U} \cdot \mathbf{W} \cdot \mathbf{W}^{-1} \cdot \mathbf{U}^T - 1) \cdot \mathbf{b} + \mathbf{b}'| \\ &= |\mathbf{U} \cdot [(\mathbf{W} \cdot \mathbf{W}^{-1} - 1) \cdot \mathbf{U}^T \cdot \mathbf{b} + \mathbf{U}^T \cdot \mathbf{b}']| \\ &= |(\mathbf{W} \cdot \mathbf{W}^{-1} - 1) \cdot \mathbf{U}^T \cdot \mathbf{b} + \mathbf{U}^T \cdot \mathbf{b}'| \end{aligned} \quad (2.6.10)$$

Now, $(\mathbf{W} \cdot \mathbf{W}^{-1} - 1)$ is a diagonal matrix that has nonzero j components only for $w_j = 0$, while $\mathbf{U}^T \mathbf{b}'$ has nonzero j components only for $w_j \neq 0$, since $\mathbf{b}'$ lies in the range of $\mathbf{A}$. Therefore the minimum obtains for $\mathbf{b}' = 0$, q.e.d.

Equation (2.6.7), which is also equation (2.6.6) applied associatively to $\mathbf{b}$, is thus very general. If no w_j 's are zero, it solves a nonsingular system of linear equations. If some w_j 's are zero, and their reciprocals are made zero, then it gives a “best” solution, either the one of shortest length among many, or the one of minimum residual when no exact solution exists. Equation (2.6.6), with the singular $1/w_j$'s zeroized, is called the *Moore-Penrose inverse* or *pseudoinverse* of $\mathbf{A}$.

Equation (2.6.7) is implemented in the SVD object as the method `solve`. (As in `LUdcmp`, we also include an overloaded form that solves for multiple right-hand sides simultaneously.) The argument `thresh` inputs a value below which w_j 's are to be considered as being zero; if you omit this argument, or set it to a negative value, then the program uses a default value based on expected roundoff error.

```
void SVD::solve(VecDoub_I &b, VecDoub_O &x, Doub thresh = -1.) { svd.h
Solve  $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$  for a vector  $\mathbf{x}$  using the pseudoinverse of  $\mathbf{A}$  as obtained by SVD. If positive,
thresh is the threshold value below which singular values are considered as zero. If thresh is
negative, a default based on expected roundoff error is used.
    Int i,j,jj;
    Doub s;
    if (b.size() != m || x.size() != n) throw("SVD::solve bad sizes");
    VecDoub tmp(n);
    tsh = (thresh >= 0. ? thresh : 0.5*sqrt(m+n+1.)*w[0]*eps);
    for (j=0;j<n;j++) { Calculate  $U^T B$ .
        s=0.0;
        if (w[j] > tsh) { Nonzero result only if  $w_j$  is nonzero.
            for (i=0;i<m;i++) s += u[i][j]*b[i];
            s /= w[j]; This is the divide by  $w_j$ .
        }
        tmp[j]=s;
    }
    for (j=0;j<n;j++) { Matrix multiply by  $V$  to get answer.
        s=0.0;
        for (jj=0;jj<n;jj++) s += v[j][jj]*tmp[jj];
        x[j]=s;
    }
}

void SVD::solve(MatDoub_I &b, MatDoub_O &x, Doub thresh = -1.)
Solves  $m$  sets of  $n$  equations  $\mathbf{A} \cdot \mathbf{X} = \mathbf{B}$  using the pseudoinverse of  $\mathbf{A}$ . The right-hand sides are
input as  $b[0..n-1][0..m-1]$ , while  $x[0..n-1][0..m-1]$  returns the solutions. thresh as above.
{
    int i,j,m=b.ncols();
    if (b.nrows() != n || x.nrows() != n || b.ncols() != x.ncols())
        throw("SVD::solve bad sizes");
    VecDoub xx(n);
    for (j=0;j<m;j++) { Copy and solve each column in turn.
        for (i=0;i<n;i++) xx[i] = b[i][j];
        solve(xx,xx,thresh);
        for (i=0;i<n;i++) x[i][j] = xx[i];
    }
}
```

Figure 2.6.1 summarizes the situation for the SVD of square matrices.

There are cases in which you may want to set the value of `thresh` to larger than its default. (You can retrieve the default as the member value `tsh`.) In the discussion since equation (2.6.5), we have been pretending that a matrix either is singular or else isn't. Numerically, however, the more common situation is that some of the w_j 's are very small but nonzero, so that the matrix is ill-conditioned. In that case, the direct solution methods of *LU* decomposition or Gaussian elimination may actually give a formal solution to the set of equations (that is, a zero pivot may not be encountered); but the solution vector may have wildly large components whose algebraic cancellation, when multiplying by the matrix $\mathbf{A}$, may give a very poor approximation to the right-hand vector $\mathbf{b}$. In such cases, the solution vector $\mathbf{x}$ obtained by *zeroing* the small w_j 's and then using equation (2.6.7) is very often better (in the sense of the residual $|\mathbf{A} \cdot \mathbf{x} - \mathbf{b}|$ being smaller) than *both* the direct-method

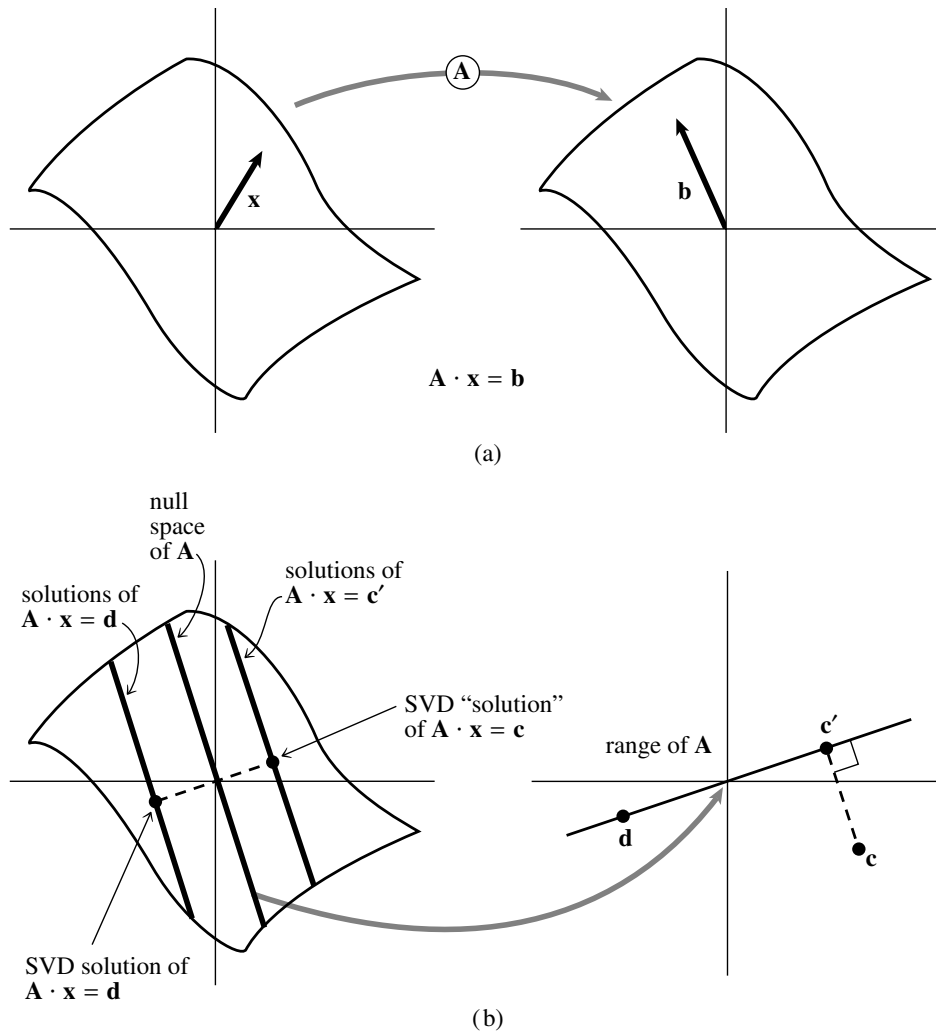


Figure 2.6.1. (a) A nonsingular matrix A maps a vector space into one of the same dimension. The vector x is mapped into b , so that x satisfies the equation $A \cdot x = b$. (b) A singular matrix A maps a vector space into one of lower dimensionality, here a plane into a line, called the “range” of A . The “nullspace” of A is mapped to zero. The solutions of $A \cdot x = d$ consist of any one particular solution plus any vector in the nullspace, here forming a line parallel to the nullspace. Singular value decomposition (SVD) selects the particular solution closest to zero, as shown. The point c lies outside of the range of A , so $A \cdot x = c$ has no solution. SVD finds the least-squares best compromise solution, namely a solution of $A \cdot x = c'$, as shown.

solution and the SVD solution where the small w_j 's are left nonzero.

It may seem paradoxical that this can be so, since zeroing a singular value corresponds to throwing away one linear combination of the set of equations that we are trying to solve. The resolution of the paradox is that we are throwing away precisely a combination of equations that is so corrupted by roundoff error as to be at best useless; usually it is worse than useless since it “pulls” the solution vector way off toward infinity along some direction that is almost a nullspace vector. In doing this,

it compounds the roundoff problem and makes the residual $|\mathbf{A} \cdot \mathbf{x} - \mathbf{b}|$ larger.

You therefore have the opportunity of deciding at what threshold `thresh` to zero the small w_j 's, based on some idea of what size of computed residual $|\mathbf{A} \cdot \mathbf{x} - \mathbf{b}|$ is acceptable.

For discussion of how the singular value decomposition of a matrix is related to its eigenvalues and eigenvectors, see §11.0.6.

2.6.3 SVD for Fewer Equations than Unknowns

If you have fewer linear equations M than unknowns N , then you are not expecting a unique solution. Usually there will be an $N - M$ dimensional family of solutions (which is the nullity, absent any other degeneracies), but the number could be larger. If you want to find this whole solution space, then SVD can readily do the job: Use `solve` to get one (the shortest) solution, then use `nullspace` to get a set of basis vectors for the nullspace. Your solutions are the former plus any linear combination of the latter.

2.6.4 SVD for More Equations than Unknowns

This situation will occur in Chapter 15, when we wish to find the least-squares solution to an overdetermined set of linear equations. In tableau, the equations to be solved are

$$\begin{pmatrix} \mathbf{A} \end{pmatrix} \cdot \begin{pmatrix} \mathbf{x} \end{pmatrix} = \begin{pmatrix} \mathbf{b} \end{pmatrix} \quad (2.6.11)$$

The proofs that we gave above for the square case apply without modification to the case of more equations than unknowns. The least-squares solution vector $\mathbf{x}$ is given by applying the pseudoinverse (2.6.7), which, with nonsquare matrices, looks like this,

$$\begin{pmatrix} \mathbf{x} \end{pmatrix} = \begin{pmatrix} \mathbf{V} \end{pmatrix} \cdot \begin{pmatrix} \text{diag}(1/w_j) \end{pmatrix} \cdot \begin{pmatrix} \mathbf{U}^T \end{pmatrix} \cdot \begin{pmatrix} \mathbf{b} \end{pmatrix} \quad (2.6.12)$$

In general, the matrix $\mathbf{W}$ will not be singular, and no w_j 's will need to be set to zero. Occasionally, however, there might be column degeneracies in $\mathbf{A}$. In this case you will need to zero some small w_j values after all. The corresponding column in $\mathbf{V}$ gives the linear combination of $\mathbf{x}$'s that is then ill-determined even by the supposedly overdetermined set.

Sometimes, although you do not need to zero any w_j 's for *computational* reasons, you may nevertheless want to take note of any that are unusually small: Their corresponding columns in $\mathbf{V}$ are linear combinations of $\mathbf{x}$'s that are insensitive to your data. In fact, you may then wish to zero these w_j 's, by increasing the value of `thresh`, to reduce the number of free parameters in the fit. These matters are discussed more fully in Chapter 15.

2.6.5 Constructing an Orthonormal Basis

Suppose that you have N vectors in an M -dimensional vector space, with $N \leq M$. Then the N vectors span some subspace of the full vector space. Often you want to construct an orthonormal set of N vectors that span the same subspace. The elementary textbook way to do this is by Gram-Schmidt orthogonalization, starting with one vector and then expanding the subspace one dimension at a time. Numerically, however, because of the build-up of roundoff errors, naive Gram-Schmidt orthogonalization is *terrible*.

The right way to construct an orthonormal basis for a subspace is by SVD: Form an $M \times N$ matrix $\mathbf{A}$ whose N columns are your vectors. Construct an SVD object from the matrix. The columns of the matrix $\mathbf{U}$ are your desired orthonormal basis vectors.

You might also want to check the w_j 's for zero values. If any occur, then the spanned subspace was not, in fact, N -dimensional; the columns of $\mathbf{U}$ corresponding to zero w_j 's should be discarded from the orthonormal basis set. The method `range` does this.

QR factorization, discussed in §2.10, also constructs an orthonormal basis; see [3].

2.6.6 Approximation of Matrices

Note that equation (2.6.1) can be rewritten to express any matrix A_{ij} as a sum of outer products of columns of $\mathbf{U}$ and rows of $\mathbf{V}^T$, with the “weighting factors” being the singular values w_j ,

$$A_{ij} = \sum_{k=0}^{N-1} w_k U_{ik} V_{jk} \quad (2.6.13)$$

If you ever encounter a situation where *most* of the singular values w_j of a matrix $\mathbf{A}$ are very small, then $\mathbf{A}$ will be well-approximated by only a few terms in the sum (2.6.13). This means that you have to store only a few columns of $\mathbf{U}$ and $\mathbf{V}$ (the same k ones) and you will be able to recover, with good accuracy, the whole matrix.

Note also that it is very efficient to multiply such an approximated matrix by a vector $\mathbf{x}$: You just dot $\mathbf{x}$ with each of the stored columns of $\mathbf{V}$, multiply the resulting scalar by the corresponding w_k , and accumulate that multiple of the corresponding column of $\mathbf{U}$. If your matrix is approximated by a small number K of singular values, then this computation of $\mathbf{A} \cdot \mathbf{x}$ takes only about $K(M + N)$ multiplications, instead of MN for the full matrix.

2.6.7 Newer Algorithms

Analogous to the newer methods for eigenvalues of symmetric tridiagonal matrices mentioned in §11.4.4, there are newer methods for SVD. There is a divide-and-conquer algorithm, implemented in LAPACK as `dgesdd`, which is typically faster by a factor of about 5 for large matrices than the algorithm we give (which is similar to the LAPACK routine `dgesvd`). Another routine based on the MRRR algorithm (see §11.4.4) promises to be even better, but it is not available in LAPACK as of 2006. It will appear as routine `dbdscr`.

CITED REFERENCES AND FURTHER READING:

- Numerical Recipes Software 2007, “SVD Implementation Code,” *Numerical Recipes Webnote No. 2*, at <http://www.nr.com/webnotes?2> [1]
- Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 9.[2]
- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §8.6 and Chapter 12 (SVD). QR decomposition is discussed in §5.2.6.[3]
- Lawson, C.L., and Hanson, R. 1974, *Solving Least Squares Problems* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 1995 (Philadelphia: S.I.A.M.), Chapter 18.
- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I.10 by G.H. Golub and C. Reinsch.[4]
- Anderson, E., et al. 1999, LAPACK User's Guide, 3rd ed. (Philadelphia: S.I.A.M.). Online with software at 2007+, <http://www.netlib.org/lapack>.[5]
- Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of *Lecture Notes in Computer Science* (New York: Springer).
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §6.7.[6]

2.7 Sparse Linear Systems

A system of linear equations is called *sparse* if only a relatively small number of its matrix elements a_{ij} are nonzero. It is wasteful to use general methods of linear algebra on such problems, because most of the $O(N^3)$ arithmetic operations devoted to solving the set of equations or inverting the matrix involve zero operands. Furthermore, you might wish to work problems so large as to tax your available memory space, and it is wasteful to reserve storage for unfruitful zero elements. Note that there are two distinct (and not always compatible) goals for any sparse matrix method: saving time and/or saving space.

We considered one archetypal sparse form in §2.4, the band-diagonal matrix. In the tridiagonal case, e.g., we saw that it was possible to save both time (order N instead of N^3) and space (order N instead of N^2). The method of solution was not different in principle from the general method of LU decomposition; it was just applied cleverly, and with due attention to the bookkeeping of zero elements. Many practical schemes for dealing with sparse problems have this same character. They are fundamentally decomposition schemes, or else elimination schemes akin to Gauss-Jordan, but carefully optimized so as to minimize the number of so-called